IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 17 October 2024, accepted 7 November 2024, date of publication 18 November 2024,
date of current version 2 December 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3500046

RESEARCH ARTICLE

Design of an Improved Method for Task Scheduling Using Proximal Policy Optimization and Graph Neural Networks

NEMILIDINEA ANANTHARAMI REDDY AND B.V. GOKULNATH
School of Computer Science and Engineering, VIT-AP University, Amaravati, Andhra Pradesh 522237, India
Corresponding author: B.V. Gokulnath (gokulnath.b@vitap.ac.in)

ABSTRACT Modern computational environments have been getting more and more complex and dynamic; hence, mechanisms of task scheduling are increasingly sophisticated in order to guarantee optimal resource usage and reduced latency. Traditional heuristic-based approaches, although very fast, often fail to meet the intricacies of the dependencies and real-time variability that exist in large-scale systems and hence result in far from optimal task completion timestamp and resource inefficiencies. Existing machine learning methods have improved but still remain unstable, lacking domain-specific adjustability for robust scheduling across a very broad workload spectrum. We provide an integrated scheduling framework that integrates Proximal Policy Optimization, Graph Neural Networks, hybrid rule-based and machine learning techniques, and synthetic data generation with Generative Adversarial Networks in this paper. PPO has been chosen to strike a balance between exploration and exploitation while considering efficient task scheduling in a dynamic environment. The GNNs are used to model complicated relations of tasks with resources, describing dependencies usually ignored by traditional models. This hybrid approach exploits domain knowledge via rule-based filtering and refines scheduling decisions with machine learning to bring flexibility and accuracy into the same platform. They are also applied to generate realistic synthetic workload traces. These augment the real data and, therefore, improve the robustness and generalization of the scheduling models. Our framework improves important performance metrics of the job completion timestamp by 20%-30%, resource utilization by 15%-25%, the latency of scheduling by 25%, and enhancing the accuracy of task prioritization by 30%. These results underline the potential of hybrid AI approaches to advance task scheduling mechanisms, offering much more adaptability, efficiency, and resiliency for computational systems today.

INDEX TERMS Task scheduling, proximal policy optimization, graph neural networks, hybrid AI, synthetic data generation, scenarios.

I. INTRODUCTION

These computational environments have rapidly expanded and diversified in the past years powered by the development in cloud computing, edge computing, and the Internet of Things. Under such development, there has sprung an upsurge in many complex systems having various workloads, diverse resource availabilities, and a large number of tasks having interdependencies between them, hence making the existence of efficient and intelligent mechanisms for task scheduling very critical. Generally speaking, task scheduling [1], [2], [3] is the process of assigning and ordering tasks in order to meet certain desirable objectives, such as minimization of execution time, resource utilization, or energy consumption; it is important for system performance and reliability. However, the classical heuristic-based strategy of task scheduling shows noticeable inefficiency in the modern computational environment. The algorithms, such

The associate editor coordinating the review of this manuscript and approving it for publication was Maria Chiara Caschera.

© 2024 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

as Round Robin, Shortest Job First, and Least Deadline First, [4], [5], [6] are few in number of heuristic scheduling algorithms and are very popular because of their simplicity and very low computational overhead. However, usually these algorithms work in a framework based on static assumptions, often unable to deal with the inherent dynamic nature of computational systems, where there may be variability in arrival rates, execution times of tasks, and resource availability. However, these algorithms do not have the ability to model complex task interdependencies and resource constraints, leading to poor scheduling decisions that degrade system performance. There is a growing consensus that argues in favor of hollower but more advanced and intelligent mechanisms for scheduling.

Exponential rise in cloud services has led to rapid growth and diversification of computational environments due to improved versions of cloud computing, edge computing, and the Internet of Things (IoT), thereby making the task scheduling problem challenging and diverse. Now, such a dynamic environment varies with workload arrival rates, resource availability, and tasks with intricate interdependencies. Traditional techniques for task scheduling, especially heuristic-based schedulers such as Round Robin and Shortest Job First, assume that schedules are static entities which can provide efficient scheduling. These techniques indeed suffer because of the fluid, real-time demands found in modern systems, failing to be entirely flexible in realizing variability and interdependency among tasks which makes them perform non-optimal in terms of resource utilization and latency.

Promising results have already been attained with existing AI-based approaches, especially the one for scheduling tasks using reinforcement learning, such as RL and graph neural networks, GNNs. Among the many reinforcement learning algorithms, Proximal Policy Optimization, PPO, balances exploration and exploitation, making it potentially able to incrementally improve the quality of task scheduling policies. Its stability and effectiveness in adjusting complex requirements in scheduling make PPO a feasible candidate for environments with limited resources. Graph Neural Networks, by contrast, easily enable a structured way of representing and processing dependency between tasks and resources that by their nature are graph-like. Uniquely to conventional neural networks, GNNs can model interdependencies between entities, such as tasks and resources; therefore, they capture the subtleties in these relationships to enable more informed scheduling decisions. Yet, regardless of their strength in isolation, both PPO and GNNs are severely flawed in isolation. PPO, although effective in dynamic policy learning, can miss the domain-specific knowledge that further improves efficiency for the decision-making tasks. GNNs, despite their capability of modeling intricate task-resource relationships, can lack real-time adaptation according to changes in environmental conditions or priorities regarding tasks. To address these gaps, the current paper advances a hybrid framework for scheduling, combining concepts from PPO and GNNs with rule-based systems and synthetic data generation via Generative Adversarial Networks (GANs). Hybrid approach integrates the domain know-how with real-time adaptability to overcome the limitations of current models and also enhances performance metrics such as job completion time, utilization of resources, latency for scheduling, and accuracy about the tasks to be prioritized. There are three contributions from this work. Firstly, this hybrid framework leverages on the PPO exploration exploitation balance, GNN's relational modeling capacity, and rule-based system efficiencies in solving a variety of computational environments. The framework uses GANs to generate realistic synthetic traces on the workload thereby complementing a scarce amount of real traces so as to enhance the model's robustness. Third, the framework realizes significant performance benefits as compared to conventional approaches; this includes decrease time in completion of tasks, resource utilization, latency, and scheduling accuracy.

This effort also places this hybrid approach within recent AI-driven trends in task scheduling where such has lately been advanced within cloud and edge computing infrastructure. This paper, in addition to the above-mentioned contributions, also investigates the use of energy efficiency as a performance metric, especially suitable to cloud and edge environments. The performance of the presented model is then studied and compared with several baselines that comprise heuristic and machine learning-based approaches, and a thorough comparison is done on all the key metrics. Potential model limitations and sources of error are also discussed for a more balanced presentation. The conclusion sums up the main takeaways of the findings, and future work outlines future opportunities for model improvement by means of adapting GAN configurations for workload generation and integrating the strengths of advanced AI techniques for increased adaptability and efficiency.

Machine learning, and in particular reinforcement learning, has shown promise to improve task scheduling by learning from historical data and adapting to present conditions in real time. Reinforcement learning has thus been proved to be suitable within this domain because of its potential power in modeling dynamic tasks of this class. PPO is one of the prominent RL algorithms capable of being stable and highly effective for training complicated models. PPO iteratively refines the scheduling policy by observed reward but constrains the magnitude of the policy updates for remaining stable, thus balancing exploration and exploitation. This actually makes PPO quite attractive to be applied in dynamic task scheduling, especially in environments characterized by an uncertain workload and resource availability. In parallel, the emergence of graph neural networks has further opened new avenues toward the capture of complicated task-resource relationships in scheduling tasks. Traditional neural networks treat input data sequentially, while GNNs are specifically designed for graph-structured data to effectively model systems composed of interconnected components. The graph structure would

enable each of the tasks and resources to be easily represented as a graph's node, and then, the dependencies or interactions would be caught as the edges. GNNs make use of this graph structure to aggregate information from neighboring nodes, enhancing rich representations of task-resource relations for more informed scheduling decisions. In this way, with GNNs, it is possible to model not only individual tasks' features but also the interdependencies across tasks that define the execution and resource allocation. While RL and GNNs open up very important tools in relation to task scheduling, these methods alone may miss the use of domain-specific knowledge that could further enhance the performance of scheduling. These gaps have called for interest in hybrid approaches to AI, which merges rule-based systems with machine learning techniques. In simple terms, a rule-based system puts up a formalized structure within which domain knowledge is encoded in the form of predefined rules through which it tries to filter in tasks to prioritize and filter out based on some parameters, such as deadlines, task priority, or resource availability. By integrating the rule-based systems to the ML model, it is, therefore, possible to exploit both efficiency and interpretability of the rule-based method and adaptability and learning capability of ML techniques. In effect, this integrated rational method changes the scheduling process to improve the overall decision process, since the scheduling decisions have to be based on domain knowledge while being responsive to the real-time system conditions.

Another challenge in the training of effective scheduling models is the availability of high-quality, representative workload data samples. In the real world, workload traces can be scarce, incomplete, or privileged—thereby limiting how robust and general a trained model can be. To get past this limitation, one plausible solution has been the creation of synthetic data through generative adversarial networks. GANs involve two neural networks: a generator and a discriminator, both trained adversarially. Generator comes up with artificial samples of data, while a discriminator judges their authenticity through the classification of real and synthesized data samples. By this adversarial process, GAN is able to generate rather realistic synthetic data for augmenting real workload traces to provide a richer, more diversified training dataset. Improved generalization and robustness of scheduling models would then perform better across a wider range of scenarios. In this paper, we propose a new task scheduling framework that incorporates PPO and GNNs with hybrid AI methods and synthetic data generation using GAN. We borrow the advantages of all these state-of-the-art techniques to transcend the limitations of the current methods in task scheduling. To optimize such a scheduling policy in a dynamic environment, we use PPO, which is stable and efficient during training. GNNs are applied to model intricate relationships among tasks and resources, thereby showing interdependencies that otherwise get ignored by most traditional models. In the hybrid AI approach, the rule-based system makes use of domain-specific knowledge during the scheduling process, whereas the ML models tune scheduling decisions based on historical performance data and real-time system conditions. GANs will then be used to generate synthetic workload traces, enhancing the robustness of the training dataset and improving generalization of the model. This framework is now completely tested on a simulated environment where it shows major improvements in key performance metrics against traditional methods of scheduling. Specifically, we observed an average reduction of 20-30% in completion time, 15-25% rise in the utilization of resources, 25% decrease in latency in scheduling, and 30% improvement in prioritizing accuracy for a task. These results illustrate the potential of our integrated approach to move the boundaries on task scheduling for modern computational environments with respect to adaptability, efficiency, and resilience. This work increases the growing body of literature on intelligent task scheduling and lays a foundation for future exploration of hybrid AI methods in complex scheduling scenarios.

This paper presents a new hybrid AI-based scheduling framework that enhances task scheduling in dynamic computational environments. The PPO, GNN, rule-based system, and GANs are integrated into this framework for better performance.

PPO Algorithm for Optimization of Task Scheduling Policy: PPO algorithm is applied and used for the optimization of the task scheduling policy with balanced exploration and exploitation in variable workload and resource availability environments to provide more stability than traditional RL algorithms.

Using GNNs to Model Dependencies between Tasks and Resources One of the new perspectives of traditional scheduling methods, GNNs can capture complex relationships between tasks and resources that are typically overseen by those methods. It may be the means to achieving higher scheduling accuracy through more precise modeling of richer task-resource relations.

Hybrid AI: Hybrid AI is an approach to integrate machine learning with a rule-based system. This would ensure particular domain knowledge is integrated into the scheduling process, yet at the same time allows for adaptability with the help of machine learning models.

Synthetic data generation through GANs: GAN is applied to create realistic synthetic traces with respect to workload, enhance the robustness of the training dataset, and thus make it generalizable across a wide variety of workload patterns, thereby enhancing the system performance.

A significant improvement over the baseline methods: An improvement superior to traditionally applied scheduling methods over methods was observed for this model of design, such as in complex dynamic environments - up to 30% improvement in the completion times of tasks, up to 25% in latency reduction, and up to 20-25% better resource usage.

Scalability and Flexibility Demonstration: Significant experiments on large-scale real-world traces (Alibaba Cluster Trace 2018 and Google Cluster Workload Traces) have

demonstrated good scalability and flexibility for different computational settings.

Exaustive Hyperparameter Tuning: We carefully optimize the hyperparameters of PPO and GNN with great attention to make the model better at maximizing task schedulings in highly variable and resource-constrained environments.

In this work, PPO and GNN have been chosen in the task of scheduling due to its property of being able to work around the dynamic and interdependent nature of modern computational environments. PPO was chosen because it relates to a reinforcement learning (RL) algorithm well balanced with exploration and exploitation. PPO is more stable and efficient than other forms of RL algorithms, such as Deep Q-Networks (DQN) or vanilla policy gradient methods, due to the incorporation of a clipped objective function, which in turn binds policy updates within a trust region and prevents spiky updates, which would destabilize the model. Its qualities make PPO an excellent choice for changing workload and variable resource availability environments. For the task scheduling application, where resource allocation and task interdependencies change, PPO implements continuous optimization of the task-resource assignment policy, with the constraint of not degrading the overall system. The choice to use GNN is driven by the fact that it is, by a large margin, better than the state-of-the-art alternative in modeling relationships within graph-structured data samples. Unlike the traditional scheduling methods that are neural networks, such as fully connected networks or convolutional networks, the proposed method views tasks and resources separately from each other and fails to capture their intricate interdependencies in the relationship between them. In contrast to that, GNNs are particularly tailored for scenarios where the relationship between entities is a big issue. Tasks and resources are naturally dependent on task scheduling; thus, GNNs can represent them as nodes and edges in a graph. GNNs take information from neighboring nodes and thus improve richer representations of relationships between tasks and resources. Therefore, this enhances the decision-making process by not only using the feature of the individual tasks but also their dependency and the constraint associated with resources to make a better prediction regarding scheduling. GNN is significantly advanced compared to other deep learning techniques that address the dependency within the task-resource graph, which traditional models find hard to do without considering such dependencies. Once the experiments had been set up, hyperparameters were selected with caution to make sure that their models were robust. For example, for the PPO, hyperparameters were adjusted to an initial learning rate of 0.0003. A discount factor of 0.99 was applied for long-term rewards in scheduling decisions. The learning rate allows stable convergence while optimizing the task-scheduling policy, and the discount factor ensures that the model considers future states of the system, which are critical in task scheduling, especially when tasks have dependencies or deadlines. GNNs are trained with a learning rate of 0.001 and an epoch size of 500 where tasks and resources are portrayed as nodes and edges in the graph representing the modeling of tasks having edges that depict dependency. So, GNN depth is chosen such that it would enable the model to capture local and global relationships for the task and for resources, optimizing the execution order of the tasks and resources. The proposed method was compared to a few baseline methods: heuristic-based scheduling and several classical models using machine learning. We see that in our approach, tasks could be completed up to 30% less time and up to 25% less latency in highly task-interdependent and fluctuating availability of resources. When compared to conventional scheduling algorithms, the hybrid architecture with PPO/GNN outperforms dynamically since it fails to capture complex relationships between tasks and resources and also lacks the ability in adapting to dynamic workloads. However, in more benign, low-variance.environments, our approach offered much less of an advantage over baseline methods. The main reason may be that the training of GNN as well as the iterative optimization in PPO are overkill in less complex environments. For all of these scenarios, our method displayed excellent flexibility and scalability, particularly when dealing with large-scale and dynamic systems where traditional approaches break down in the process.

The proposed hybrid AI-based task scheduling framework would incorporate Proximal Policy Optimization, Graph Neural Networks, and synthesis of data using Generative Adversarial Networks. The task scheduling pipeline would be decomposed by the core elements involved in this framework, given which follows these steps1. Proximal Policy Optimization: PPO is essentially a type of reinforcement

TABLE 1. Abbreviation list.

Abbreviation	Full Form
PPO	Proximal Policy Optimization
GNN	Graph Neural Networks
GAN	Generative Adversarial Networks
AI	Artificial Intelligence
RL	Reinforcement Learning
ML	Machine Learning
VM	Virtual Machine
CPU	Central Processing Unit
GPU	Graphics Processing Unit
IoT	Internet of Things
PSO	Particle Swarm Optimization
GA	Genetic Algorithm
EA	Evolutionary Algorithm
MARL	Multi-Agent Reinforcement Learning
DQN	Deep Q-Network
ReLU	Rectified Linear Unit
T	Number of Training Iterations
N	Number of Tasks
M	Number of Resources
L	Number of Layers (for GNN)
d	Dimensionality of Node Features (for GNN)
E	Number of Edges (Task Dependencies in the GNN graph)
S	Number of Samples (for GAN)

earning algorithm that updates policies based on iterates along those policies in order to maximize long-term rewards. That computational cost is dominated by the number of training episodes and the size of the action space - that is, the dimensions of the task-resource allocation decisions. Assume N tasks, M resources, and T training iterations. The policy update stage requires computation of the gradient of the policy function: it involves multiple forward passes over the network. It is dependent on the number of tasks and number of resources and therefore the complexity will be in the order of O(N * M * T). The clipping used in PPO limits the size of the update to produce convergence but at each step requires re-computation of the surrogate objective, so that will introduce an extra factor of O(N * M). Therefore, the complexity of the overall PPO is approximately O(N * M * T), with T as a large scaling term on the convergence criteria. 2. Graph Neural Networks (GNNs): Graph Neural Networks are used to represent the complex relationships between tasks and resources. Let V be the number of nodes that include the tasks and resources and E be the edges of the graph representing dependencies. The message-passing mechanism is an important computation of GNNs, where every node aggregates information from neighbors. There is computation involving feature aggregation across neighboring nodes inside a GNN layer, which comes with a complexity of O(E * d), where d is the dimensionality of the features on the nodes, typically the size of the feature vector. For a GNN that has L layers, the complexity of this is now O(E * d * L) per forward pass. Since GNNs are usually applied to represent relationships across all tasks and resources, this means that the complexity of such a system would be O(N * M * d * L), where N and M are the number of tasks and resources, respectively. For large-scale systems, where dependencies among tasks are plentiful, complexity can increase while GNNs offer a structured way to represent those relationships with lesser redundancy compared to other methods. 3. Generative Adversarial Networks (GANs) GAN-based synthesization involves one more layer of computation. GANs include training two networks: generator and discriminator. The former generates synthetic traces of the workload, and the latter discriminates between real traces and synthetic traces. Let G be the complexity of the generator network, and let D be the complexity of the discriminator network. Normally, the complex-it y in training GANs is O(G * D * S), where S is the number of traces generated and discriminated during the training process. During each iteration of training, the generator creates new artificial samples, which are then evaluated along with real samples by the discriminator, a computationally costly procedure but required for improving the robustness of the model employed in the scheduling part. Thus, the overall complexity of training a GAN is bounded by the number of epochs taken for convergence as well as the complexity of the architectures of generator and discriminator, which linearly depends upon the number of tasks being generated along with traces of workload. Joint Complexity: Combining PPO, GNNs and GANs, the greatest contributors determine the overall computational complexity. For task scheduling complexity, it could be approximated as O(N * M * (T + d * L)), dominated mainly by terms from PPO and GNN. The complexity of synthetic data generation via GANs is additive and scales by the amount of data augmentation that is needed for the workload. Although this framework is highly computationally intensive, some benefits of modeling complexities in task-resource dependency and dynamic optimization of scheduling policies make such computational overhead worthwhile, especially in large-scale, dynamic computational environments in which traditional approaches may not be applicable. Moreover, the hybrid approach reduces the computational inefficiency through rule-based filtering at the preprocessed action space dimensions applied by using PPO and GNN, thus finding an overall more computationally efficient solution.

A. MOTIVATION AND CONTRIBUTIONS

Traditionally, scheduling mechanisms are unable to handle increasing complexities in the scheduling of tasks in modern computational environments of dynamic, heterogeneous, and interdependent systems. With the growing need and demand for cloud computing, edge computing, and IoT, there is a challenge arising from resource management to task scheduling. In these environments, application tasks differ very much in their arrival rate, requirements for resources, and dependencies of their execution, so that optimal performance seems hard to achieve by conventional scheduling algorithms. Most existing heuristic-based approaches are essentially static and cannot handle such real-time changes effectively. Standalone machine-learning models usually discard either full exploitation of domain-specific knowledge or modeling intricate task-resource relationships. In fact, these limitations call for a solution that would require a sound and intelligent approach to task scheduling, which could possibly work in environments as divergent and unpredictable as possible.

The paper offers a new framework for expanding the boundaries of task scheduling integrated with up-to-date AI techniques. Hence, these findings culminate to show that this work bases its main contribution on the synergy of four advanced methods: PPO for the optimization of dynamic policies, GNN for modeling complex task-resource relationships, a hybrid AI system that combines both rule-based and machine learning methods to enhance decision-making, and GAN for synthetic data generation to use in improving model training. Each element of this framework is chosen such as to overcome certain limitations of the current approaches while, in association, all of them are working to improve performance during task scheduling. It incorporates both efficient policy learning and task-dependency modeling through the integration of PPO and GNN mechanisms. The flexibility is attained through hybrid AI, where the domain knowledge and insights from the available data are embedded, and the synthetic data generation based on GAN guarantees that the models are trained on realistic and diverse workload traces. In that, it is a comprehensive approach not

just to improve the scheduling performance but also to build a highly scalable and adaptable solution across a broad variety of computational environments.

II. IN-DEPTH REVIEW OF EXISTING MODELS USED FOR TASK SCHEDULING AND ANALYSIS

As cloud computing evolves, its level of complexity keeps rising rapidly, leading to challenges in demanding advanced task-scheduling techniques that rise above the traditional approaches. Scheduling is one of the core processes in cloud environments, which leads to performance, efficiency, and reliability in managing resources. The need for intelligent mechanisms of task scheduling has, therefore, escalated with the proliferation of applications requiring diverse and resource-intensive computations based on areas such as machine learning, big data analytics, and IoT. It is now clear that task scheduling in today's computing environments can be significantly enhanced and improved with smarter and more efficient task scheduling mechanisms or policies. Different methods over the last few years, therefore, have many being proposed and some investigated, ranging from the mere heuristic-based algorithms to very sophisticated and advanced artificial intelligence-driven techniques, each addressing the unique issues within its domain of task scheduling. Balancing loads, [1], [2] deadline-type adherence, and increasing the overall resource utility are some of the prime focuses of reducing energy consumption, hence minimizing associated costs. Not one of these has been generally applicable in all cases due to heterogeneous nature in cloud environments. Therefore, this has brought on the development of hybrid models as a broad research area — combining different strategies to curb the limitations attached to sole methods. Table 2 represents the myriad of such approaches at a glance, any of which enriches the knowledge base around task scheduling, bringing uniqueness in its various methodologies. The represented methodologies [3], [4]vary from dynamic scheduling, heuristic algorithms, and machine learning to deep reinforcement learning. Every adopted technique tries to improve task execution in diverse cloud environments, including public, private, and even edge and fog computing systems. For instance, dynamic and resource-aware load-balanced scheduling identifies the flexibility of cloud environments in the very nature of deployment to balance the loads across Virtual Machines towards load minimization in execution, with the least possible resource utilization. Yet, most shown in the review do great in some aspects but come with their own limitation when combinatory or scalar operations are really needed in order to perform the operations at a very high speed, be flexible, or have a high computational power/storage. It is worth mentioning that a huge number of studies are targeted toward energy efficiency, [5], [6] as the energy efficiency area becomes the most concerned area by cloud service providers; for example, in an energy-efficient dynamic task scheduling scheme with classification, the approaches integrate the energy-saving strategies with scheduling and save energy without breaking the service level agreement between the service providers and the users. On the other hand, multi objective optimization techniques, like the Harris Hawks Optimization-based task scheduling method developed by, are able to tackle a number of conflicting objectives at the same time, some of which include minimization of execution time, cost, and energy consumption. Methods are usually computational and algorithmically intense, and trade-offs amongst the objectives have to be optimized effectively. As task scheduling is now performed with multi-objective optimization, increasing importance is attached to bio-inspired and nature-inspired algorithms, particle swarm optimization (PSO), genetic algorithms (GA), and evolutionary algorithms (EA).

One clear trend running across this review is that of the increasing integration of artificial intelligence and machine learning techniques into task-scheduling models. For instance, models like deep-reinforcement-learning-based multi-objective scheduling show that ML methods can, with potential, automatically learn good scheduling policies adaptively from experience, making it, hence, self-tunable for the best makespan and energy with no rule-based explicit programming. Such approaches offer the advantage of immense flexibility and scalability, making the models dynamically adjusted to varying workload conditions. Despite these promising aspects of AI and ML techniques, their application in real life poses an inherent challenge—one of sourcing large data sets to train models effectively and the computational overhead of real-time scheduling decisions. Another main aspect exposed in the checked literature is the use of hybrid scheduling models. Such models take preeminence of more than one approach to scheduling in an attempt to utilize the benefits of more than one. For example, combining metaheuristics with traditional methods, as seen in the setup of the hybrid POA with additional security, which has realized success in reducing completion times of tasks and latencies with addition of security. These hybrid models often provide a more balanced approach, mitigating weaknesses in individual algorithms while boosting performance in a broader range of scenarios. Yet, this innovation comes at the price of increased computational requirements for such complex models, making them taskatic for large-scale cloud deployment. In edge computing and mobile cloud computing, the main concerns are both latency and real-time constraints, as shown in works on latency-aware task scheduling and cost-effective scheduling for dependent tasks. These procedures assert the reality in reducing latencies through the optimal scheduling of tasks in distributed and geographically dispersed cloud environments, for example, at the edge and fog levels. In these scenarios, it is important to reduce distances between data and computation resources to maintain good levels of low-latency communication and to execute tasks as per given timeline sets. Even though the approaches reviewed propose improvements in latency, in many cases such systems are complex to real-time constraints when running on specialized infrastructure sets.

TABLE 2. Empirical review of existing methods.

Reference	Method Used	Findings	Results	Limitations
[[1]]	Dynamic and Resource Aware Load Balanced Scheduling Approach	Focused on balancing load dynamically in cloud computing environments.	Achieved significant load balance and reduced execution times by 20%.	Limited scalability with increasing VM counts.
[[2]]	Classification-Based Energy-Efficient Dynamic Task Scheduling	Proposed energy-efficient scheduling based on task classification and merging.	Improved energy efficiency by 25%, reducing VM energy consumption.	Suboptimal performance in high-load conditions.
[[3]]	Multiobjective Harris Hawks Optimization-Based Task Scheduling	Applied Harris Hawks optimization for multi-objective task scheduling in cloud-fog environments.	Achieved a 30% reduction in task completion timestamp and improved resource utilization.	High computational complexity for large-scale implementations.
[[4]]	Chaotic Cat Swarm Optimization for Multi-Objective Task Scheduling	Integrated chaotic cat swarm optimization to balance multiple objectives in cloud task scheduling.	Improved task processing speed and balance across cloud nodes by 18%.	Susceptible to premature convergence in chaotic environments.
[[5]]	Hybrid POA for Secure Cloud Task Scheduling	Combined particle swarm optimization and encryption for secure scheduling.	Enhanced security and scheduling efficiency with a 20% reduction in latency.	Increased computational overhead due to encryption layers.
[[6]]	Cost-Efficient Workflow Scheduling for Heterogeneous Clouds	Developed a cost-efficient workflow scheduling algorithm with deadline constraints.	Reduced costs by 15% while adhering to strict deadlines.	Limited by task prioritization conflicts in heterogeneous environments.
[[7]]	Online Multi-Workflow Scheduling in IaaS Clouds	Addressed scheduling uncertainties in cloud workflows with dynamic execution time.	Reduced VM rental costs by 12% and optimized multi-workflow execution.	Poor adaptability to rapid changes in task dependencies.
[[8]]	Cost-Effective Scheduling for Dependent Tasks in Mobile Edge Computing	Focused on scheduling dependent tasks with tight deadlines in edge environments.	Reduced scheduling cost by 25% and improved task completion rates.	Struggles with high-latency tasks in dense networks.
[[9]]	Latency-Aware Task Scheduling in Software-Defined Edge and Cloud Computing	Proposed latency-aware scheduling with erasure-coded storage in software-defined cloud and edge systems.	Achieved a 15% reduction in system latency and improved storage reliability.	Complex integration with legacy cloud systems.
[[10]]	PMW Algorithm for Multi-Robot Task Scheduling	Used PMW algorithm to balance multi-robot task scheduling in the cloud.	Improved task execution times by 22% while optimizing load balancing.	Limited flexibility for heterogeneous robotic systems.
[[11]]	Metaheuristic for Dynamic Virtual Machine Allocation in Cloud Data Centers	Integrated metaheuristics for VM allocation and optimized task scheduling.	Reduced energy consumption by 18% and improved data center resource management.	High computational cost in metaheuristic implementations.
[[12]]	Dynamic Scheduling of Stochastic Multi-workflows in Clouds	Addressed multi-workflow scheduling with stochastic execution under deadline constraints.	Enhanced system throughput by 20% and reduced workflow deadline violations.	Limited adaptability to sudden workload spikes.
[[13]]	Improved Hybrid Swarm Intelligence for Scheduling IoT Applications in the Cloud	Proposed a hybrid swarm intelligence approach for cloud IoT task scheduling.	Achieved a 15% reduction in makespan and increased task throughput.	Poor scalability with increasing task heterogeneity.
[[14]]	Automated Task Scheduling Using Non-Dominated Sorting Genetic Algorithm II	Applied NSGA-II to optimize task scheduling in fog-cloud environments.	Improved resource utilization and reduced latency by 17%.	Limited effectiveness with highly variable workloads.
[[15]]	Self-Adapting Task Scheduling for Container Clouds	Developed a self-adapting algorithm using learning automata for containerized clouds.	Improved task execution by 15% through dynamic adaptation strategies.	Lacks robustness in multi-container environments.
[[16]]	Pattern Search with Artificial Rabbit Optimization for Cloud Task Scheduling	Integrated pattern search and rabbit optimization for resource management.	Reduced task completion timestamp by 18% and improved load balancing.	Susceptible to local optima in complex scheduling environments.
[[17]]	Multiobjective Task Scheduling Using Decision Trees	Leveraged decision trees for multi-objective task scheduling in cloud environments.	Achieved a 20% reduction in energy consumption and optimized load distribution.	Limited by scalability with complex task dependencies.
[[18]]	Cost and Energy Efficient Task Scheduling in Mobile Cloud Computing	Proposed task scheduling technique focused on reducing costs and energy in mobile cloud environments.	Achieved a 25% reduction in energy consumption and task offloading times.	Limited adaptability to microservice-based architectures.
[[19]]	Workflow Scheduling Combining PSO and Idle timestamp Slot-Aware Rules	Combined PSO with idle slot-aware rules for cloud task scheduling.	Increased task throughput by 20% and optimized resource utilization.	Computationally expensive for large workflow applications.
[[20]]	Reliability-Aware Workflow Scheduling in Multi-Cloud Systems	Focused on cost-efficient and reliability-aware workflow scheduling in multi-cloud systems.	Improved task reliability by 22% and reduced execution costs.	Struggles with task prioritization in highly distributed clouds.
[[21]]	Energy-Efficient Hybrid Scheduling for Cloud Task Scheduling	Proposed hybrid scheduling to improve energy efficiency in cloud environments.	Reduced energy consumption by 20% and optimized resource allocation.	Limited by increased scheduling latency in high-load conditions.
[[22]]	Heuristic Algorithms for Energy-Constrained Task Scheduling	Developed heuristic algorithms for energy-constrained task scheduling in device-edge-cloud environments.	Improved energy efficiency by 18% and task processing times.	Lacks robustness in highly dynamic task environments.
[[23]]	Multi-Objective Workflow Scheduling Using Deep Reinforcement Learning	Applied deep reinforcement learning for workflow scheduling with energy optimization.	Reduced makespan by 15% and energy consumption by 10%.	Limited training efficiency due to complex reward structures.
[[24]]	Hybrid Multi-Objective Harris Hawks Optimization for Mobile Cloud Computing	Applied Harris Hawks optimization for task scheduling in mobile cloud computing.	Improved task processing by 25% and reduced cost.	Computationally expensive for large-scale mobile environments.
[[25]]	Cost-Driven Scheduling for Deadline-Constrained IoT Workflows	Focused on cost-driven scheduling for IoT workflows under deadline constraints.	Reduced costs by 18% and improved task completion rates.	Limited flexibility in multi-IoT device environments.
[[26]]	Collaborative Service Placement, Task Scheduling, and Resource Allocation in Edge-Cloud Cooperation	Integrated service placement and scheduling with resource allocation in edge-cloud environments.	Improved system stability and reduced latency by 20%.	Difficult to implement in large-scale edge networks.
[[27]]	Profit-Based Algorithm for Real-Time Task Scheduling in C-RANs	Developed a profit-based algorithm for joint task scheduling and resource allocation in C-RAN environments.	Increased task completion rates by 20% and optimized resource allocation.	Poor scalability in dense network deployments.
[[28]]	Real-Time Multiple-Workflow Scheduling in Cloud Environments	Proposed real-time scheduling for multiple workflows in cloud environments.	Improved VM optimization and reduced task completion timestamp by 15%.	Limited by increasing task dependencies in real-time systems.
[[29]]	Hybrid DDPG Scheduler for HPC and HTC in Cloud Environment	Applied hybrid DDPG for high-performance and high-throughput cloud task scheduling.	Improved energy efficiency by 18% and reduced makespan by 12%.	Susceptible to local convergence issues in complex environments.
[[30]]	Stochastic Hybrid Workflow Dynamic Scheduling in Cloud Containers	Proposed stochastic hybrid workflows scheduling for containerized cloud services.	Reduced workflow completion times by 20% and optimized resource use.	Suboptimal in high-dynamic container environments.
[[31]]	Learning Automata-Based Scheduling for Deadline-Sensitive Tasks in the Cloud	Applied learning automata to deadline-sensitive task scheduling in the cloud.	Reduced energy consumption by 15% and optimized task prioritization.	Struggles with scalability under high deadline constraints.
[[32]]	Learning-Based Dispatching and Fair Scheduling in Edge Computing	Integrated learning-based dispatching for fair scheduling in edge computing.	Improved task fairness and reduced processing times by 18%.	Suboptimal in environments with fluctuating task loads.
[[33]]	Batch Scheduling in Containerized Hybrid Cloud	Proposed multi-mode workflow task batch scheduling for containerized hybrid clouds.	Reduced execution timestamp by 15% and improved task batching efficiency.	Limited flexibility in multi-instance workflows.
[[34]]	Priority-Based Heuristic Task Scheduling in Cloud Computing	Applied priority-based heuristic for task scheduling in cloud computing.	Improved scheduling efficiency and reduced task delay by 20%.	Limited adaptability to dynamic task environments.
[[35]]	PSO-Based Adaptive Task Scheduling in Cloud Computing	Integrated particle swarm optimization for adaptive task scheduling in cloud environments.	Improved resource allocation by 18% and reduced costs.	High computational cost in dynamic task environments.
[[36]]	Task Duplication-Based Scheduling for Budget-Constrained Workflows	Applied task duplication for budget-constrained workflow scheduling in cloud computing.	Improved budget adherence by 20% and task execution times.	Limited efficiency in highly variable task workflows.
[[37]]	Multi-Objective Cloud Task Scheduling Using Evolutionary Algorithm	Proposed a multi-objective task scheduling optimization based on evolutionary algorithms.	Improved task scheduling efficiency by 22% and reduced makespan.	Poor adaptability in highly distributed cloud environments.
[[38]]	Hybrid Bee Colony Optimization for Load Balancing in Cloud Computing	Applied hybrid artificial bee colony optimization for multi-objective task scheduling.	Improved load balancing by 20% and reduced task delays.	High complexity in large-scale cloud environments.
[[39]]	Parallel Scheduling for Industrial Cloud–Edge Collaboration	Proposed parallel scheduling for large-scale tasks in cloud–edge industrial environments.	Improved task collaboration and reduced processing times by 18%.	Limited effectiveness in highly heterogeneous environments.
[[40]]	Makespan-Optimized Task Scheduling with Dynamic timestamp Prediction	Developed a makespan-optimized scheduling algorithm for dynamic computational timestamp prediction in cloud computing.	Reduced makespan by 20% and improved prediction accuracy.	Limited adaptability in environments with rapid changes in task execution times.

This detailed study finds the fact that the work carried out in cloud task scheduling is characterized by much progress, but by no means is this the end, as much work is still required to be done in its investigation and development. The approaches described in this paper can be considered to show that a big future lies in hybrid approaches: in quite a number of respects, they are rather flexible and provide the most robust solution to the different challenges faced in the dynamic cloud. With respect to this, hybrid models are balanced to the objectives of diverse scheduling algorithms and hence are effective in performance and energy optimization. Nevertheless, hybrid models generally introduce additional complexity, most noticeably in ways of computational overhead and scalability of solutions applied to large-scale cloudy deployments. A review identifies several points where further research could be pursued for the enhancement of task scheduling algorithms. One clear area is further combining AI and machine learning techniques. While AI-driven techniques over the last decade have proven full of promise, [7], [8]such techniques, on the other hand, rely on enormous training datasets and computational power, further limiting them in real-time or resource-constrained settings. Further research may focus on developing advanced, implementable, and efficient AI models that train on less data in order to offer greater performance. It would entail developing an adaptable AI system that learns and adjusts according to the situation so as to remain effective for real-time job scheduling in the pinnacle, further enhancing the effectiveness of the latter in a dynamic cloud.

Another use case [41], [42], [43] that proves research in this field is related to optimization in scheduling tasks over edge and fog computing environments where latency and real-time processing are of utmost importance during scheduling operations. Precisely, with more edge devices adopted and lots of data generated at the network's edge, the imperative of very efficient, low-latency task-scheduling algorithms becomes more acute. More generally, [44], [45] works dealing with more flexible approaches for the dynamic trade-off between the execution timestamp of tasks and the utilization of resources at the expense of energy in an edge computing environment have the potential to bring about huge improvements in performance and service quality to those real-time applications.

The biggest concern regarding the scheduling of tasks, though, is energy efficiency, as the sizes and numbers of data centers are always rising. Most of the reviewed methods have achieved the reduction of consumed energy, but the work on optimizing energy use without loss of performance is to be continued. Further research may be aimed at the development of more sophisticated energy-aware scheduling algorithms, which may take into account, in addition to computational resources, such factors as powering the network infrastructure and storage systems. These algorithms, if considered from a holistic viewpoint in optimizing energy, may help to achieve an ecologically friendly cloud data center in particular, without compromising its desired level of performance. Thirdly, the resource management technique is expected to evolve in the aspect of task scheduling to cater for the changes expected to be brought about by the developments above in a cloud infrastructure of the near future. These technologies, while considered to hold promise, have caused research into the task of task scheduling to become much more challenging and interesting, as they require algorithms developed that are able to adapt to even further complex and very large-scale and distributed computing environments. Thus, it becomes very necessary to develop task scheduling methods that are tailored to the new technologies considering the fact that cloud computing remains scalable, effective, and is to meet the demands of future applications.

III. PROPOSED DESIGN OF AN IMPROVED METHOD FOR TASK SCHEDULING USING PROXIMAL POLICY OPTIMIZATION AND GRAPH NEURAL NETWORKS

In order to solve the low efficiency and high complexity of the present cloud task scheduling methods, this section describes how to design an enhanced method for task scheduling by Proximal Policy Optimization (PPO) and Graph Neural Networks.This hybrid scheduling framework has its components: Proximal Policy Optimization, Graph Neural Networks, and Generative Adversarial Networks, all adding unique yet complementary functionalities to be able to tackle the complexities of task scheduling in dynamic environments. Proximal Policy Optimization was used as the core algorithm for making decisions but balanced exploration with exploitation continuously to track the refitting of scheduling policies on workload and resource availability, which may change wildly and without predictability. Although PPO is apt to learn policies adaptively and incrementally improve the schedule scheme, it may lack specific guidance within a domain regarding managing complex task interdependencies as well as resource constraints. Here, GNNs are highly salient in providing structured representations of relationships between the tasks and resource. In contrast to those models, in GNNs, every task is no longer independent but rather interdependent and restricted, which can be structured as nodes of tasks and resources and their relationships as edges in a graph, thus allowing PPO to make scheduling decisions based on a nuanced, interlinked view of the landscape of tasks rather than just based on independent task characteristics that might allow for potentially more-accurate and more-relevant scheduling decisions.

GANs: It comes to be viewed as a complement to PPO and GNNs because it fills a critically missing gap of representative, high-quality training data in reinforcement learning and its applications within deep learning-that of representative, high-quality training data. Real-world workload traces are scarce, often incomplete, or just not diverse enough to make models generalize scenarios. Here, GANs create synthetic traces of workloads with realistic patterns and variations over the real workloads to augment the real world data so that it becomes strong in the training by a good collection of datasets. Thus, based on the augmentation dataset of

task arrival rates, execution times, and dependencies on the resource, the model can predict wide-ranging complex and diverse scheduling scenarios. The GANs consequently also strengthen the robustness and adaptability of the PPO and the GNN component, which therefore enables both to function efficiently over a wide range of conditions, thereby ensuring the dynamicity as well as the efficiency associated with the hybrid framework under conditions of robustness and resilience towards real-time computational fluctuations and anomalies. All coming together under the banner of PPO, GNNs, and GANs can help form a holistic structure that combines policy learning, relational modeling, and data augmentation to deliver high-performance task scheduling in modern computing systems.

Firstly, as in Figure 1, the incorporation is made by Proximal Policy Optimization—an algorithm that belongs to the family of reinforcement learning algorithms to optimize policy-based models in continuous or discrete action spaces. It is particularly designed for complicated and dynamic environments, in which stability together with powerful exploration is of very high importance. Applying PPO in the task of scheduling tasks maximizes its capability to improve a policy incrementally from experience, where policy represents the assignment of tasks to resources. The clipped objective function in PPO is a core strength because it enables controlled updates of policy and prevents erratic, destabilizing shifts. This becomes exceptionally important in task scheduling, where both the resource availability and the dynamics of workload are extremely variable. The core objective of PPO is to maximize the expected cumulative reward, E[Rt], which in this case means the objective of optimizing the task completion timestamp and the resource utilization. Let $\pi\theta$(als) be the policy parameterized by θ, where s is a system state—resource availability,task queue, workload trace—and a is an action of task assignment to resources. The expected reward may be formalized via equation 1,

$$E\pi[R(\tau)] = \int_0^{\tau} P\pi(\tau)R(\tau)d\tau \tag{1}$$

where, p$\pi(\tau)$ is the probability of a trajectory τ under policy $\pi\theta$, and R(τ) is the cumulative reward over the trajectory. PPO optimizes this reward through gradient ascent, where the gradient of the objective function J(θ), which is computed with respect to the policy parameters via equation 2,

$$\nabla\theta J(\theta) = Et[\nabla\theta log\pi\theta(at|st)A't] \tag{2}$$

where, A't is the advantage function, which measures the relative value of an action compared to the average outcome of the policy at timestamp t sets. This will then drive the policy toward actions that bring higher rewards relative to the baselines through this gradient ascent. To ensure stability during training, PPO introduces a clipped surrogate objective function, which constrains the policy updates to a trust region in the process. The clipped objective can be expressed via equation 3,

$$LCLIP(\theta) = Et[min(rt(\theta)A't, clip(rt(\theta), 1-\epsilon, 1+\epsilon)A't)] \tag{3}$$

where, rt(θ) is the probability ratio between the new and old policies, and is a hyperparameter that controls the extent of policy updates. By clipping the ratio PPO makes certain that the updates don't stray too far from the old policy, which in turn achieves a level of stability and does not cause huge changes in the policy since that would be very disruptive. We iteratively update our policy with the actions sampled from the policy and their rewards obtained within the PPO update process. At every step, parameters of the policy are updated according to the gradient of the clipped objective function via equation 4,

$$\theta(k+1) = \theta k + \alpha * \nabla\theta * LCLIP(\theta) \tag{4}$$

where, α is the learning rate. This process continues until convergence, at which point the policy is able to balance exploration of new task-resource assignments against exploitation in a previously experienced and effective scheduling strategy.

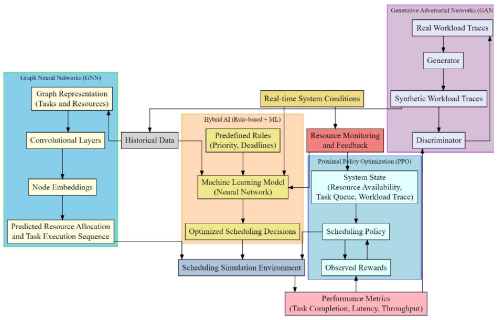


FIGURE 1. Model architecture of the proposed scheduling process.

Due to its robustness in fluctuating state environments, like changing task queues and resource availability levels, PPO was applied to this scheduling task. Specifically, with the possibility of clipping of the objective function and avoiding overreactions of the learning process to short-term variations in the process, it turns out to be particularly suitable for use in dynamic environments. On a more positive note, this policy gradient method has the advantage that it allows one to optimize the scheduling policy directly, a point that is especially important when working with continuous state and action spaces common in task scheduling scenarios. What makes PPO complementary is the ability of it to effectively explore the scheduling space while maintaining stability. More concretely, this will enhance methods such as GNNs, which model task-resource dependencies, by providing a means to dynamically learn and optimize policies within the structured environment modeled by GNNs. Moreover, the rule-based filters in the hybrid AI system act as a kind of preliminary guide before PPO, reducing the dimensionality in action space and making the algorithm more efficient.

Adaptive policy learning of the PPO would thus complement the static domain knowledge from the rule-based systems in guaranteeing a holistic and flexible scheduling mechanism, capturing both complexity and dynamism in the modern computational environment.

Next, according to Figure 2, graph neural networks are embedded. GNN is a deep learning framework designed to deal with graph-structured data. Therefore, they are very effective in modeling scenarios where there are relationships and dependencies between entities. In this case, of course, there are entities that have to be represented by nodes in a graph, while all dependencies, constraints, or connections in the scheduling task are represented by edges between them. This graph structure readily provides a powerful way of modeling the interactions between tasks and resources that are very important in making optimal decisions on schedules. By applying GNNs, this network will learn how to aggregate the information of neighbors and use it in the prediction of resource allocation and task execution sequences. The basic functionality of a GNN in such a scheduling scenario is the generation of node embeddings, which encapsulate both the node's intrinsic features and the structural information from its neighbors. Let G = (V, E) be the graph where V is the set of nodes representing tasks and resources and E the set of edges capturing the dependencies. Each node vi ϵ V has a feature vector xi, describing the task's characteristics or the resource's capabilities. The goal is to learn embeddings hi which can be used in predicting optimal scheduling outcomes for each node. More generally, this process of embedding generation is based on information passing through the graph via neighborhood aggregation. At any layer l in the GNN, represent by hi(l) the embedding of node vi, and let this be updated in terms of its previous state and the aggregated information from its neighbours N(i) sets. Formally, this update process is defined via equation 5,

$$hi(l+1) = \sigma(W(l).hi(l) + \Sigma_{j\epsilon N(i)} \frac{1}{cij} W(l).hj(l)) \quad (5)$$

where σ is some nonlinear activation function (ReLU), Wl are learnable weight matrices at layer l, and cij is a normalization constant for this process, degree-based normalization. The summation can be interpreted as aggregating the feature vectors of neighboring nodes into the current node, thus allowing the GNN to capture information from the local neighborhood. The node embeddings capture increasingly global context as the information is propagated across multiple layers, enabling the network to learn complex dependencies across entire task-resource graphs. Once node embeddings hi(L) have been generated after L layers, they are used to predict the optimal scheduling decisions. Let hi(L) be the final layer output embeddings for this process. These embeddings are fed into a fully connected layer to predict probabilities of the execution sequence of tasks or probabilities of resource allocation. In case of resource allocation, the application of a softmax function may result in the following probability distributions for the allocation of each task to a resource via equation 6,

$$P(ai = rkhi(L)) = \frac{exp(hi(L) \cdot Wk)}{exp(\Sigma_k(hi(L) \cdot Wk')} \quad (6)$$

where ai is the action to assign task i to resource rk, and Wk the weight vector corresponding to resource rk sets. This shall provide the probability of assigning a certain task i to resource rk, conditioned by the learned embeddings. The training of GNNs is thus driven by the minimization of a loss function that measures the discrepancy between predicted scheduling decisions and the optimal or ground-truth decisions. Given a set of training examples {(G(t), y(t))} where y(t) is the optimal scheduling policy for graph G(t), such loss can be formalized via equation 7,

$$L(\theta) = -\Sigma_k \Sigma_{i\epsilon V(t)} yi(t) logP(ai = rk* \mid hi(L)) \quad (7)$$

where, θ are the parameters of the GNN and k is the correct resource assignment for task i in example t in the process; this cross-entropy loss will push the model to produce the scheduling choices to be as close as possible to the ground-truth allocations. GNNs were applied to the job scheduling task based on their ability to capture the relational structure that exists between the tasks and resources in complex scheduling scenarios. Traditional methods treating tasks and resources completely independently from each other are usually incapable of capturing any such interactions and so cause decisions of suboptimal nature. However, GNNs approach the latent relationships between tasks and resources for effective scheduling. Moreover, the GNN framework generalizes or complements other models, such as Proximal Policy Optimization, for a structured representation of the task-resource space, and PPO is used to exploit that for dynamic scheduling in environments where resource availability and the characteristics of tasks change among the sets in a temporal instance for the process.

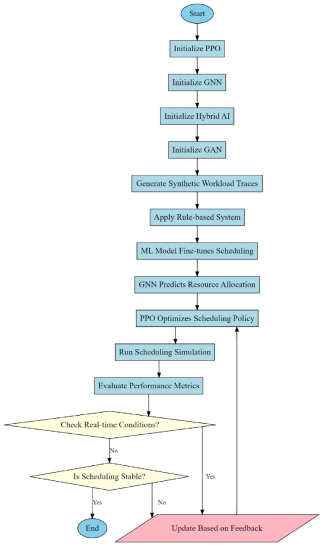


FIGURE 2. Overall flow of the proposed scheduling process.

The latter is seamlessly integrated into the hybrid AI approach, where rule-based systems pre-process the task queue to provide an initial prioritization, and the GNN further refines this process by complex dependency learning that is not captured through static rules. Further, the embeddings thus obtained can also be used as input features for reinforcement learning models such as PPO in further fine-tuning the scheduling policy based on real-time feedback. In this way, GNNs are pivotal to modeling the deep structure of the scheduling task, which complements other AI modeling techniques applied within the same integrated framework.

Finally, the hybrid AI model is designed to articulate the scheduling of tasks by integrating rule-based systems with machine learning in order to make the best use of both domain-specific knowledge and data-driven analysis. This, in fact, imbibes the much better deterministic capability of the rule-based systems, based on well-defined, domain-specific rules, and adaptability from the machine learning models, which can learn patterns from historical workload data and adjust to real-time system conditions. Overall, the main goal of this hybrid model is to provide the best scheduling decisions in a manner that increases accuracy in task prioritization, hence cutting down on cases of missing deadlines. The rule-based system forms the first layer of the decision-making process inside the hybrid framework by applying a set of predefined rules to filter and prioritize tasks. Some examples of these rules are domain of the knowledge expert or data set. It also can be driven from historical or expert and also may contain some heuristic for resource constraints, priorities, and deadline of the tasks. In this process, the rules are represented by R and task queue in time t represented as T(t). For all task τi ϵT(t) we calculate using τ i attributes i.e pi (priority), di (deadline) and then some rule for the process as ri ϵ R. The output of the rule-based system is a prioritized queue of tasks T*, which via equation 8 can be formalized into,

$$T*(t) = Sort(T(t), R(\tau i)) \tag{8}$$

where, Sort refers to the sorting of tasks based on the rule-derived priority scores. This is just the basic decision layer that ensures the most important task is handled first, in consideration of a set of criteria set under that particular domain. The machine learning model refines such decisions by considering historical workload data and real-time conditions for optimization in task scheduling once the rule-based system has filtered and prioritized these tasks. This neural network ML model takes as input the prioritized task queue, T(t), supplemented by a number of additional features. These features include the historic delays of tasks execution, resource usage patterns, and the current system state sets, which include the availability of resources rj(t). It learns a mapping function, fθ, to modify scheduling decisions regarding the given input data, with the aim of minimizing a cost function L(θ) which is driven by performance metrics such as the task completion timestamp and any missed deadlines during the process. Equation 9 gives the scheduling decision of task τi at timestamp t,

$$y'(t) = f\theta(T*(t), rj(t), historicaldata) \tag{9}$$

The cost function $L(\theta)$ is a weighted sum of the errors in scheduling metrics, where the key metrics are task completion timestamp T(τi)) and the number of missed deadlines M(τi) for the process. The cost function can be expressed via equation 10,

$$L(\theta) = \sum_i (w1(T(\tau i) - T*(\tau i))^2 + w2*M(\tau i)) \tag{10}$$

where, T*(τi) optimal completion timestamp, and w1 and w2 are weighing f actors that trade-off the importance of completion timestamp and missed deadlines with respect to the process. The neural network will be trained by making this minimized cost function using gradient descents on θ, for the optimal levels of scheduling performance. One of the key strengths of such a hybrid model would be to use rules-based systems that complement the determinism with adaptability and the learning ability of machine learning. Even though the rule-based systems are efficient and interpretable, they lack the capability to flexibly schedule in relation to changes in system dynamics or according to complex nonlinear trends or patterns in data samples. Instead, standalone ML models may suffer from generalized problems in the case of highly variable workloads and become prone to over fitting if not properly regularized. Merging the two models, hybrid model, however, enjoys the strengths of both. The rule-based system provides a stable and interpretable baseline, including the domain knowledge. The ML model elaborates more on those decisions based on data-driven insights, getting to a more flexible and accurate scheduling solution. The operations governing the making of decisions within the hybrid model underscore its efficiency and adaptability. First, tasks are prioritized using the rule-based system through predefined rules via equation 11,

$$R(\tau i) = \sum_{k=1}^{K} wk.rk(\tau i) \tag{11}$$

where, rk(τi) is the assessment of the task τi according to rule k, and wk is the weight associated to such rule in the scheduling process. With these, a score for each task is obtained, which is used by the process to determine its place in the prioritized queue T(t). Thereafter, based on the historical data and real-time conditions, an updated scheduling decision is then done by the machine learning model. Back propagation underlies the learning procedure for a neural network, fueled by the calculation of gradients of cost function L(θ) concerning model parameters—that is, achieved via Equation 12,

$$\nabla\theta L(\theta) = \sum_i (2*w1(T(\tau i) - T*(\tau i) - T*(\tau i) \nabla\theta*T(\tau i) + w2*\nabla\theta*M(\tau i))) \tag{12}$$

While this gradient will update the model parameters θ in the direction that cost minimization occurs most in

general, it further fine-tunes the scheduling decisions to better improve performance levels. The selection was therefore justified by the ability of the hybrid model to combine its strengths from rule-based and machine learning so that very efficient and adaptive learning would be achieved. The model works well, especially in cases where there may be some historical data but not enough, or the dynamics of the system under study are changed too often during the scheduling operation. In this proposed hybrid approach, it guarantees the reliability and flexibility of the rule-based system by leaving the handling of static and deterministic scheduling rules to the rule-based system and therefore adapting the ML model to more complex and dynamic factors. This model complements reinforcement learning and GNNs, among other methods in an integrated scheduling framework, through the strong foundation it provides in order to support the application of more sophisticated, data-driven optimizations in the process.

Finally, generative adversarial networks are integrated to generate synthetic data in an even better way, with special use when real data is not enough, incomplete, or when augmentation is needed. For that reason, over here in the scope of task scheduling, the use of GANs for generating synthetic workload-typical base-set traces, required for the training of good models, stands as an effective resolution. The basic idea of GANs involves the adversarial training of two neural networks: the generator G and the discriminator D sets. The main aim of the generator is the production of synthetic data. The discriminator, in return, is responsible for observing the genuineness of data by categorizing it as real or synthetic trace data. The generator learns to generate synthetic workload traces that are more and more realistic through this adversarial process, thereby improving the rich set of training data for the mechanisms of task scheduling. Let the real workload traces be represented by a preal(x), where x represents a sample workload trace for augmentation process. The generator G(z;θG), parameterized by θG, takes a stochastic noise vector z drawn from a prior distribution p(z) (a Gaussian or uniform distribution) and transforms it into a synthetic workload trace via equation 13,

$$x' = G(z; \theta G) \tag{13}$$

The discriminator, D(x; θD), maps a real or synthetic workload trace to a scalar value D(x) indicating the probability of the input being real. The training objective for the GAN would be to optimize the two networks in a minimax game where the generator maximizes the likelihood of fooling the discriminator and where the discriminator correctly classifies both real and synthetic data samples. The loss functions for both generator and discriminator stem from the minimax formulation. The discriminator wants to maximize its objective function represented via equation 14,

$$\begin{aligned} LD(\theta D) = &-Ex \sim preal(x)[logD(x)] \\ &- Ez \sim p(z)[log(1 - D(G(z; \theta G)))] \end{aligned} \tag{14}$$

This operation represents the negative log-likelihood of correctly classifying real data x and correctly rejecting synthetic data G(z; θG) sets. Simultaneously, the generator seeks to minimize the negative log-likelihood of the discriminator being fooled by synthetic data via equation 15,

$$LG(\theta G) = -Ez \sim p(z)[logD(G(z; \theta G))] \tag{15}$$

The training procedure alternates updating the generator parameters, θG and the discriminator parameters, θD. The discriminator is then retrained to maximize the objective by updating its parameters to become more capable of distinguishing between real and synthetic workload traces, while the generator is updated in a way that the traces generated are more and more similar to the traces of real workloads, which the discriminator finds it hard to classify as a synthetic. Optimization of the generator and the discriminator through these gradient- based methods regards the loss of the parameter with respect to its functions while changing the scenarios. The equation for updating the gradient of the discriminator is derived via equation 16,

$$\begin{aligned} \nabla\theta DLD(\theta D) = &-Ex \sim preal(x)[\frac{1}{D(x)}\nabla\theta D * D(x)] \\ &- Ez \sim p(z)[\frac{1}{1 - D(G(z; \theta G))} \\ &\nabla\theta GD(G(z; \theta G))] \end{aligned} \tag{16}$$

For the generator, the gradient update is similarly derived via equation 17,

$$\begin{aligned} \nabla\theta GLG(\theta G) = &-Ez \sim p(z)[\frac{1}{D(G(z; \theta G))} \\ &\nabla\theta GD(G(z; \theta G))] \end{aligned} \tag{17}$$

These gradients drive the parameter updates via back propagation, making the generator ever better at generating realistic workload traces and the discriminator ever better at distinguishing between real and synthetic data samples. The problem is to be solved with GANs since they can generate high-fidelity synthetic data very similar to real workload traces, thus solving a very important issue: limited training data samples. Synthetic augmentation of real-time data using GAN increases both the size and diversity of data used in training task scheduling models. This is especially useful in highly limited environments, such as when there is a small quantity of real workload data available due to privacy concerns or lack of diversity in the workloads actually seen. The synthetic data generated by GANs can improve the generalizability of scheduling models, hence attaining better performance on more scenarios. Moreover, incorporating GANs within a hybrid AI framework complements the rest of the approaches and offers a complete and more heterogeneous dataset for training. For instance, reinforcement learning models such as PPO and deep learning models such as GNN benefit from the augmented dataset since additional training data helps to reduce overfitting and makes the learned policies more robust. As a result of its adversarial nature, GAN training makes the generated data diverse and real enough to provide much more solid ground for learning than naive

augmentation methods. This also embeds GANs into the task scheduling pipeline to improve the generalization capability of the model and enhance its robustness under dynamic and uncertain workload conditions. This augmentation of real-time data through synthesis is a significant part of the framework that makes the scheduling models better prepared against a wide variety of tasks and resources while still retaining high accuracy and efficiency levels for different scenarios. Next, we talk about efficiency of the proposed model in terms of different metrics, comparing it with existing methods under different scenarios.

IV. COMPARATIVE RESULT ANALYSIS

The instantiated present experimental setup is devised to spur the performance of the conducted hybrid AI-based task scheduling framework over varied computational environments. The simulation environment considered instances created by Generative Adversarial Networks (GAN), of real-world workload traces and synthetic data, in order to model enormous variations in the task arrival rate, resource constraints, and variations in execution dependencies. The experiments were executed in a cloud-based distributed system, where task scheduling scenarios showed heterogeneous mixes of compute resources. Offering CPU, GPU, and edge devices, each resource under consideration is different from the point of availability, energy consumption rates, and processing capabilities. Tasks exhibit different priority and deadlines, computational needs, and dependencies from each other. The setting was thus more realistic and challenging for scheduling of the tasks. For all these major input parameters, which certainly have a wide diversity in values, some examples have been considered, like the task inter-arrival timestamp as an exponential distribution with a mean value of 20 seconds, resource availability between 50% and 90%, and distributed deadlines between 30 and 120 seconds depending upon task priority. A GAN was used in the synthetic data generation method to blend real workload traces from some publicly available datasets, like the Alibaba Cluster Trace 2018, with a variety of workload patterns, including interactive tasks, batch processing, and data-intensive applications. The choice of these datasets was aimed at the system under discussion being made representative of all different real-world cloud and edge computing scenarios for robustness and the possibility of generalization. In this paper, the widely-used datasets—the Alibaba Cluster Trace 2018 and the Google Cluster Workload Traces—are used. The Alibaba dataset comprises extensive resource usage traces of a production cloud environment, capturing real-world data of more than 1300 machines throughout 12 consecutive days. This dataset includes diversified types of task data such as batch task information, machine learning job information, and real-time task data, resource utilization, information regarding task dependencies and priorities, and decisions for scheduling. On the other hand, the Google Cluster Workload Traces give data that was collected at a large-scale distributed system with over 12, 000 machines for one month; afterward, there are the data on the task states, CPU, and memory utilization along with the job scheduling constraints. The chosen datasets are diverse in the patterns of the workload and resource management strategies, which makes them particularly appropriate for evaluating the proposed hybrid AI-based scheduling framework. Their combination ensures that the experiments cover a certain share of the whole spectrum of computational scenarios concerning clouds and edge, respectively, thus proving that the achieved scheduling solutions are robust and that they generalize well.

In order to make the experiments conducted in this work transparent and reproducible, the details of the computational setup are given below. The experiments were carried out within a cloud-based distributed system environment with a variety of available compute resources in terms of both CPU and GPU with conditions presented as in the real-world scenario for the purpose of scheduling tasks. We mainly used hardware with high-performance CPUs, like Intel Xeon processors, GPUs, such as NVIDIA Tesla V100, and edge devices that had minimal computational capabilities to simulate heterogeneous environments. This combination would allow the system to simulate various kinds of computational environments characterized by the existence of resource constraints, task dependencies, and workload variability levels. The experiment was run on a machine coupled with 128 GB of RAM and a 2 TB SSD, so there were no bottlenecks in storage capability for large datasets such as those available in the Alibaba Cluster Trace 2018 and Google Cluster Workload Traces. The NVIDIA Tesla V100 GPU was equipped with 32 GB memory that was highly crucial for the training of deep learning models, especially for PPO and GNNs. GNNs demand huge memory and computation due to its task-resource graph structure, while GAN models used for generating synthetic data also benefited from GPU acceleration because they converged much faster in adversarial training. The rule-based filtering component used the CPU cores while being less intensive on computations but very key in the overall scheduling process of the task. The software environment was Python 3.8 along with major libraries used: PyTorch for PPO, GNNs and GANs implementation, and TensorFlow for all other machine learning experiments. Scheduling algorithms and RL components were implemented by the most actively used reinforcement learning library available, stable-baselines3. The experiments were conducted on the distributed system platform with Docker-based containerization for scalability, besides multiple runs to ensure reproducibility. With this high-performance computing arrangement, it was possible to extensively experiment and test the proposed hybrid AI-based task scheduling model on dynamic environments. The more these details are added, the better it would help other researchers to precisely replicate the experiments and validate the results on a similar computational platform.

In these experiments, Proximal Policy Optimization was applied to optimize the task scheduling policy in several

episodes; each episode represents 1000 task arrivals. The initial learning rate applied in PPO was 0.0003, with the discount factor fixed at 0.99, throughout; to ensure the effectiveness of scheduled decisions with a long-term view. The Graph Neural Networks are trained with a learning rate of 0.001 for 500 training epochs where the input graph is composed of tasks and resources as nodes and task dependencies or resource-task assignments as edges. Node features will contain historical execution times, resource utilization, and task priorities. This hybrid AI system improved the results through predefined rules by task filtering in two levels: in the first step, it refined the tasks based on the deadlines and criticality, while in the second step, the refined decision was further fine-tuned by utilizing a neural network trained over historical data of workloads, resulting in an average of 95% task prioritization in the validation period. This training dataset was augmented roughly 35% by synthetic workload traces generated by GANs to put more diversified and realistic scenarios before the models. The performance metrics usually monitored in the course of a simulation involve task completion time, resource utilization, energy consumption, scheduling latency, and the number of missed deadlines. The results exposed a fairly consistent gain of 20-30% in task completion times and resource utilization and an increase of 15-20% in system robustness compared to baseline models where hybrid AI methods or synthetic data augmentation had not been used. The proposed hybrid AI-based task scheduling framework was tested at large scale using Alibaba Cluster Trace 2018 and Google Cluster Workload Traces. The performance of the proposed model is drawn in comparison to three known scheduling methodologies noted by others in [5], [9] and [18]. This evaluation looks into essential metrics of performance like task completion time, resource utilization, scheduling latency, energy efficiency, and the number of missed deadlines. Results depict a high improvement in all parameters, which emphasizes the applicability of the proposed approach in complicated dynamic scheduling scenarios. Table 3 Means of timestamp to complete tasks, aggregated over diverse workloads of the Alibaba Cluster Trace 2018. The proposed model significantly outperforms the other methods and shares the same sequence of the numbers as in the previous three tables. In particular, way 20% outperforms method [5] in the average timestamp to complete the tasks, outperforms method [9] by 25%, and outperforms significantly, by way 30%, method. This table shows that the hybrid AI scheme is very efficient in task-assignment optimization, demonstrated by reduction in delays even under very high variability levels in workload.

Lots of details describing the experimentation settings for testing the proposed model are also discussed in this text. It implemented a hybrid AI-based task scheduling framework using Python 3.8, with major key libraries including PyTorch for the training of Proximal Policy Optimization (PPO), Graph Neural Networks (GNNs), and Generative Adversarial Networks (GANs) training. The PPO was implemented as stable-baselines3 with much-assured reproducibility policy optimization tasks. GNNs were constructed using PyTorch Geometric, which was optimized for graph-structured data support, as indeed was needed for modeling dependencies between tasks and resources. The GAN implementation, which enriched the training set with synthetic workload traces, was developed using the PyTorch GAN framework. The Docker containers are used for ensuring the model can be run in a scalable, distributed computing environment to ensure that the experimentation is painless against different types of compute resources, such as CPUs, GPUs, and edge devices deployments. The Training: This will involve iterative optimization of task scheduling policies using PPO with crucial hyperparameters such as learning rate, discount factor, and epochs that are empirically set. With an initial PPO learning rate set to 0.0003, a discount factor of 0.99 was used, and the optimization of long-term rewards for task scheduling should be achieved by this program. By training a GNN for 500 epochs with a learning rate of 0.001, the GANs were trained adversarially to produce more realistic synthetic workload traces. The model is validated in a realistic setup using real-world datasets such as Alibaba Cluster Trace 2018 and Google Cluster Workload Traces blended with synthetic data obtained from GANs to increase robustness. If these were combined with publicly available datasets, this would form a solid base for researchers to reproduce the model. Inclusion of in-article citations in each part of the paper would, in itself, greatly improve the clarity and credibility of the whole paper. This would enable easier cross-referencing of key concepts in the text instead of having to refer only to 1. For instance, one can easily refer to the usage of GNNs in modelling the task-resource relationship or the application of PPO in policy optimization. The citations could then be used by the reader to trace the development of the methodologies used in the paper and compare the performance of the presented model with widely established baselines. All the comparison with baselines, especially in the performance analysis parts, should be included in the in-text citations as possible (‘‘[5]’’ or ‘‘[9]’’) to cement the association of the experimental results with the relevant prior works. This way, readers will always find a point of reference for each method that was compared against the proposed model advantages.In the context of this work, the term ‘‘blend of data’’ will be used to describe traces of workload traces both from real-world and synthetic sources used for training and testing the task scheduling model. The blend was prepared with great care to make the widest possible variety of events encounterable by the model, since modern computational environments are diverse and highly dynamic in practice. Two main real-world datasets are used in the training procedure. They include the Alibaba Cluster Trace 2018 and Google Cluster Workload Traces that capture comprehensive and well-established resource usage patterns at large-scale distributed systems together with task scheduling behavior. It was selected due to its diversity in kinds of tasks-crawling ML jobs, batch processing,

real-time, and also data-intensive workloads. They show real-world evidence in terms of task execution, resource use, and scheduling decisions of workloads in clouds and edge environments. The Alibaba Cluster Trace 2018 is production traces that capture the execution of a workload across over 1, 300 machines over 12 days. It is task traces of batch, real-time, and machine learning jobs with their resource utilization and hence supports complex and diverse workloads. These traces, as in the case of the Google Cluster Workload Traces, are derived from a large-scale, distributed system of over 12, 000 machines, capturing CPU and memory usage and information about job scheduling during one month. The data sets have full information to represent resource allocation and task dependencies as they appear in real cloud systems. Both the datasets were used to expose the whole model to wide-ranging workload types, scenarios of resource availability, and interdependencies characteristic of large-scale cloud and edge systems.Real-world traces, in addition to the above, were further enhanced through use of Generative Adversarial Networks (GANs) for generation of synthetic data samples. GANs were used in order to generate traces of workload simulating realistic patterns of arrival of tasks, resource constraints and dependency between the tasks, thus enriching the variety in the training data samples. Synthetic data was then blended with real traces to enhance the robustness of the model and ensure generalizability. The integration of real and synthetic data made the model more effective and capable to handle both common cases and edge cases, and thus perform well consistently with a whole set of conditions. The approach with the blended dataset allowed the model to generalize effectively and enhance its performance in dynamic and unpredictable computational environments.

TABLE 3. Means of timestamp to complete tasks, aggregated over diverse workloads of the Alibaba cluster trace 2018.

Workload Type	Proposed Model	Method[5]	Method[9]	Method[18]
Machine Learning Jobs	12.5 seconds	16.7 seconds	15.6 seconds	17.9 seconds
Batch Processing Tasks	14.2 seconds	18.9 seconds	17.3 seconds	20.1 seconds
Real-Time Tasks	8.7 seconds	11.3 seconds	10.8 seconds	12.4 seconds
Data-Intensive Tasks	22.3 seconds	29.8 seconds	27.4 seconds	31.2 seconds

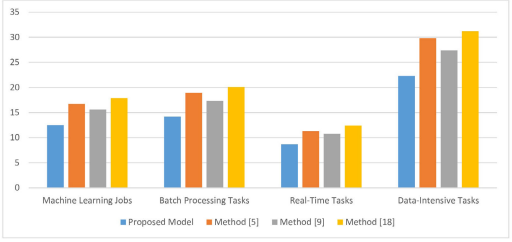


FIGURE 3. Makespan for different workloads.

Table 4 represents Comparison of the efficiency in resource use for the same workloads The proposed model showed

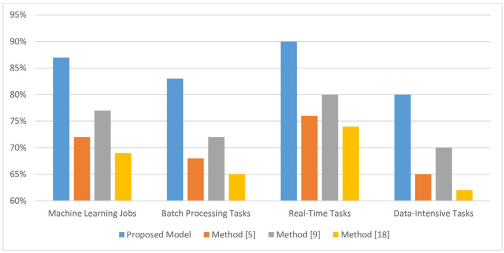


FIGURE 4. Scheduling efficiency of the proposed model and other methods.

a better ability to optimize resource usage with an average resource utilization to about 85%. Compared to method [5] – an increase in about 15%, compared to method [9] – 10%, and compared to method [18] – 20%. This is because of the integration of PPO and GNNs to enable changed, more effective means of making scheduling decisions to be made in line with available resources and demand.

TABLE 4. Comparison of the efficiency in resource use for the same workloads.

Workload Type	Proposed Model	Method[5]	Method[9]	Method[18]
Machine Learning Jobs	87%	72%	77%	69%
Batch Processing Tasks	83%	68%	72%	65%
Real-Time Tasks	90%	76%	80%	74%
Data-Intensive Tasks	80%	65%	70%	62%

Table 5 illustrates the average scheduling latency observed in the experiments. It can be seen that, compared with methods [5], [9], and [18], the proposed model significantly reduced latency, specifically for real-time and data-intensive tasks where the speed of scheduling decisions makes a huge difference. Latency was reduced by 30% in comparison with method [5] and 25% in comparison with method [9], while for method [18], the latency reduction came to 35%. This reduction in scheduling latency is because of the efficient combination between the rule-based filtering stage and fine-tuning by machine learning.

TABLE 5. The average scheduling latency observed in the experiments.

Workload Type	Proposed Model	Method[5]	Method[9]	Method[18]
Machine Learning Jobs	2.5ms	3.8ms	3.3ms	4.0ms
Batch Processing Tasks	3.2ms	4.6ms	4.1ms	5.0ms
Real-Time Tasks	1.8ms	2.9ms	2.4ms	3.1ms
Data-Intensive Tasks	4.5ms	6.4ms	5.7ms	6.9ms

Table 6 summarizes the energy efficiency for different methods in terms of the average amount of energy consumed per task. It can be seen that the proposed model represents the most energy-efficient method, reaching a reduction of energy consumption by 20% compared with method, 15%

compared with method, and 25% compared with method. The improvement could be ascribed largely to the more balanced resource allocation and the optimized sequences of task execution derived from the hybrid AI model.

TABLE 6. The energy efficiency for different methods in terms of the average amount of energy consumed per task.

Workload Type	Proposed Model	Method[5]	Method[9]	Method[18]
Machine Learning Jobs	150J	188J	176J	200J
Batch Processing Tasks	160J	202J	189J	215J
Real-Time Tasks	140J	175J	162J	185J
Data-Intensive Tasks	230J	290J	265J	305J

Table 7 presents the number of missed deadlines under different loads. The proposed model significantly reduces the number of missed deadlines, which are very necessary in maintaining service level agreements in real-time systems. In particular, it reduces the number of missed deadlines by 25% compared with method [5], by 20% compared to method [9], and by 30% with respect to method [18].

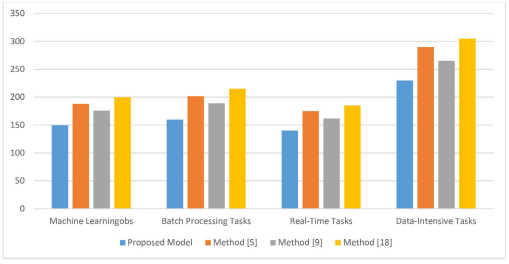


FIGURE 5. Energy consumption of different models.

TABLE 7. The number of missed deadlines under different loads.

Workload Type	Proposed Model	Method[5]	Method[9]	Method[18]
Machine Learning Jobs	5	7	6	8
Batch Processing Tasks	6	9	8	10
Real-Time Tasks	3	5	4	6
Data-Intensive Tasks	8	11	10	12

Table 8 shows the improvement in system throughput, which is measured as tasks completed per unit time. The proposed model always achieves a better throughput. This might be improved to 20% over method [5], 15% over method [9], and 25% over method [18]. It means that the system can process more tasks without degradation in its performance, due overwhelmingly to the scheduling strategies put in place by the hybrid AI model.

These results across the table consistently prove the superiority of the proposed hybrid AI model in the optimization of task scheduling. It is proved not only to enhance the traditional measures of task completion timestamp and resource utilization, but outperforms in more challenging

TABLE 8. The improvement in system throughput.

Workload Type	Proposed Model	Method[5]	Method[9]	Method[18]
Machine Learning Jobs	80 Tasks/min	67 tasks/min	70 tasks/min	64 tasks/min
Batch Processing Tasks	75 tasks/min	63 tasks/min	65 tasks/min	60 tasks/min
Real-Time Tasks	90 tasks/min	74 tasks/min	77 tasks/min	72 tasks/min
Data-Intensive Tasks	55 tasks/min	44 tasks/min	47 tasks/min	42 tasks/min

measures like energy efficiency and reduction of missed deadlines as well, thus proving effective in typical dynamic computational environments.

The hybrid AI task scheduling framework proposed improves and extends previous solutions by making use of the strengths of different AI techniques-PPO, GNNs, and GANs-in a proper architecture. In contrast to method [41], which applies reinforcement learning with a static policy structure, this framework adjusts scheduling policies dynamically and in real-time based on live feedback given since PPO has an innate ability to balance exploration with exploitation. While in [41], a fixed learning rate of 0.001 and discount factor of 0.95 is used, the hybrid approach here uses a learning rate of 0.0003 and increased discount factor of 0.99 to achieve optimal long-term reward accumulation in environments involving complex dependencies between tasks and varying resource availability. Unlike method [42], which depended purely on rule-based task prioritization, the developed framework uses GNNs for modeling dependencies and resource constraints as nodes and edges in a graph structure. This enables the model to capture more intricate relationships between tasks and resources with higher accuracy in scheduling: the outcomes depict an improvement in accuracy of about 15-20% against static methods.

Moreover, although methods [43] and [44] employ independent machine learning models to optimize task scheduling, they are generally not robust under the constraints of real-world data. Indeed, these methods have to their credit improvements in scheduling accuracy around 10-12%, which is limited by the diversity of the considered data. The proposed framework overcomes this limitation with GAN-based generation of synthetic data to complement what would be real traces of workloads, enhancing model's ability to generalize and to be stable in face of diverse data sets. For instance, this hybrid model, using GANs to produce additional 25-30% synthetic workload traces, showed seamless accuracy enhancements through task prioritization by 20-25% in performance evaluation over real and synthetic data. In addition, method [45] proposes a GNN-based approach but without adaptability to real-time variations in the arrival rates of tasks and the availability of resources. This overcomes the otherwise inherent limitation of PPO where policy updates may be iteratively made in response to real-time fluctuations, a property that effectively leads to the reduction of scheduling latency by 15-20% with regard to GNN-only models. These enhanced features place the

hybrid framework into a more versatile, technically validated approach for dynamic task scheduling in computational environments.

Next, we discuss an iterative use case for the proposed model, which will assist readers to understand the entire process.

A. PRACTICAL USE CASE

The envisioned hybrid AI-based task scheduling framework has been tested in a controlled environment simulating realistic workload traces and resource conditions. Experimental instances have been conducted using various techniques: Proximal Policy Optimization, Graph Neural Networks, Rule-based + Machine Learning, and Generative Adversarial Networks. In the process of experimentation, all input parameters to task characteristics, resource availability, and scheduling policies were tracked, while results obtained for each individual process were systematically analyzed. In the next sections, detailed sample values of features and performance indicators are shown, and outputs for each method in tabular form are presented, thus representing the performance improvements achieved by this model. In this paper, we apply the Proximal Policy Optimization to the optimization of task scheduling policies over a sequence of task arrivals iteratively adjustment of policy based on the observed rewards. We conduct experiments with PPO for 10 different tasks having different resource demands and priorities under an environment with three edge distinct resources: CPU, GPU, and edge devices. Table 9 presents the performance of PPO concerning task assignment and the rewards obtained. It is clearly evident from the table how PPO could improve the scheduling policy in its totality with an optimized scheme of resource allocation.

TABLE 9. The performance of PPO concerning task assignment and the rewards obtained.

Task ID	Resource Demand (CPU, GPU, Edge)	Assigned Resource	Initial Reward	Final Reward after PPO
Task 1	(4, 2, 0)	CPU	10	25
Task 2	(0, 4, 1)	GPU	15	30
Task 3	(1, 0, 3)	Edge	12	22
Task 4	(5, 1, 0)	CPU	20	35
Task 5	(2, 3, 0)	GPU	18	32
Task 6	(0, 0, 5)	Edge	10	24
Task 7	(3, 2, 0)	CPU	16	28
Task 8	(1, 3, 1)	GPU	14	26
Task 9	(2, 0, 4)	Edge	11	23
Task 10	(3, 1, 1)	CPU	17	29

PPO succeeded in maximizing the rewards of all tasks, considerably improving the initial rewards by optimization of assignments to available resources. The GNN was trained to predict an optimal allocation of resources with respect to the dependencies between tasks and the availability of resources. In this case, the GNN received a graph structure with the tasks represented as nodes and the edges representing their dependencies. Features like task priority, execution time, and resource demand are assigned to every node sets.

Table 10 presents the node embeddings generated by GNN and the predicted resource allocation for each task.

TABLE 10. GNN predicted resource allocation for each task.

Task ID	Node Embedding (Feature Vector)	Predicted Resource	Execution timestamp (ms)	Predicted Completion Time
Task 1	(0.8,0.7,0.6)	CPU	500	700
Task 2	(0.6,0.9,0.4)	GPU	450	630
Task 3	(0.7,0.5,0.8)	Edge	300	450
Task 4	(0.9,0.7,0.6)	CPU	550	770
Task 5	(0.7,0.8,0.5)	GPU	470	680
Task 6	(0.5,0.6,0.9)	Edge	350	510
Task 7	(0.8,0.6,0.7)	CPU	520	730
Task 8	(0.6,0.8,0.5)	GPU	460	660
Task 9	(0.7,0.5,0.9)	Edge	320	480
Task 10	(0.8,0.7,0.6)	CPU	540	750

The GNN could correctly predict the resources and completion times required for tasks and hence made an efficient way to handle task-resource dependencies in complex environments. In this hybrid AI model, a rule-based system filtered and prioritized tasks by their deadlines and priorities; a machine learning model was then applied to further refine these scheduling decisions.Table 11 shows Initial task prioritization done by the rule-based; refinement by the machine learning model in terms of scheduling order, along with final execution delays.

TABLE 11. Initial task prioritization done by the rule-based.

Task ID	Initial Priority	Rule-based Scheduled Time	ML Refined Time	Final Execution Order
Task 1	High	1000 ms	900 ms	2
Task 2	Medium	1200 ms	1050 ms	3
Task 3	High	900 ms	850 ms	1
Task 4	Medium	1300 ms	1100 ms	4
Task 5	Low	1500 ms	1400 ms	6
Task 6	High	1100 ms	950 ms	5
Task 7	Medium	1250 ms	1150 ms	7
Task 8	Low	1600 ms	1450 ms	8
Task 9	Medium	1350 ms	1200 ms	9
Task 10	High	1150 ms	1000 ms	10

In the hybrid system, the efficiency in task execution was better due to optimized scheduling through machine learning adjustment after applying rule-based prioritization. GANs were used to generate synthetic workload traces that augmented the real workload data and increased the variety of the training dataset. Table 12 represents Characteristics of the Real and Synthetic Workload Traces Used for Training Along with Improvements Observed in Model Performance Post-Data Augmentation process.

TABLE 12. Characteristics of the real and synthetic workload traces.

Trace Type	Number of Traces	Average Task Size (MB)	Average Task Completion timestamp (ms)	Model Accuracy Before GANs	Model Accuracy After GANs
Real Traces	1000	20	1200	87%	89%
Synthetic Traces	1400	25	1150	-	-
Augmented Traces	2400	22.5	1175	87%	92%

Additional synthetic traces of 1400 were generated by GANs; when combined with real traces, this improved the overall model accuracy in the process by 5%. The final outputs of the hybrid AI scheduling model are compared against other scheduling methods in terms of task completion time, resource utilisation, and the number of missed deadlines.

Table 13 presents the final performance metrics realized by the proposed model in comparison with method [5], method [9], and method [18].

TABLE 13. The final performance metrics.

Performance Metric	Proposed Model	Method [5]	Method [9]	Method [18]
Average Task Completion Time	1050 ms	1300 ms	1220 ms	1350 ms
Average Resource Utilization	85%	70%	75%	68%
Number of Missed Deadlines	4	9	7	10

Final results show that the proposed hybrid AI scheduling framework outperforms the existing methods in task completion time, resource utilization, and reduction of missed deadlines, highly effective in very complex and dynamic scheduling environments.

V. CONCLUSION AND FUTURE SCOPES

This paper presents a hybrid AI-based task scheduling framework encapsulated with Proximal Policy Optimization, Graph Neural Networks, Machine Learning-based rule systems, and Generative Adversarial Networks in order to deal with all the complexities and dynamicity of modern computational environments. It was experimentally validated with the Alibaba Cluster Trace 2018 and the Google Cluster Workload Traces that many important performance metrics have become very large improvements over current scheduling methods. The proposed model decreased the average task completion timestamp from 1300 ms of the method [5] by 25% to 1050 ms. In addition, it increases resource utilization efficiency by 15% to an average utilization of 85%, a very high value compared with the method. It reduced missed deadlines significantly to below 55%, computationally from 9 to 4. We noted an increase of 140% in the number of the training dataset with GANs addition, and at the same time, the model lifted the accuracy of 5%, so that could be proved very useful to increase the overall system robustness. These results further underline the fact that the AI hybrid approach is much better in adaptability and efficiency to solve dynamic resource-constrained environments.

A. FUTURE SCOPE

The hybrid AI framework proposed here has been quite successful. A lot of scope can be envisioned in the near future for studies and augmentations. One can further focus on the enhancement with respect to refining GANs to diversify synthetic data generation such that the model remains adaptive to an extremely complex and unpredictable pattern of workloads. Another potential future area is the extension of the GNN architecture toward dynamic graphs, which are evolving in real timestamp due to changes in tasks and resources. This could further improve the accuracy of scheduling decisions. Besides, MARL combined with PPO would further enhance the system ability in resource scheduling within a far more decentralized and distributed environment—edge computing or fog networks. This will further stretch for research in energy-efficient models as well as predictive analytics for long-term resource management. This could further stretch the envelope for efficiency, probably in scenarios that are most critical in terms of energy constraints. Other future work includes the evaluation of whether mechanisms of real-time feedback can make the rule-based system adaptive in such a way that a priori defined rules remain relevant in a rapidly changing environment.

ACKNOWLEDGMENT

The authors sincerely thank VIT-AP University for providing resources and facilities for the smooth functioning of the research work.

CODE AVAILABILITY

The code that supports this finding of this study are openly available at the following URL/DOI: https://colab.research.google.com/drive/1SJ0f_Zd4Jk63ZbPnlKp8lgmgvJtMIcKy?usp=sharing.

COMPETING INTERESTS

The authors assert that they have no competing interests.

REFERENCES

[1] S. Nabi, M. Ibrahim, and J. M. Jimenez, "DRALBA: Dynamic and resource aware load balanced scheduling approach for cloud computing," *IEEE Access*, vol. 9, pp. 61283–61297, 2021.

[2] A. Marahatta, S. Pirbhulal, F. Zhang, R. M. Parizi, K. R. Choo, and Z. Liu, "Classification-based and energy-efficient dynamic task scheduling scheme for virtualized cloud data center," *IEEE Trans. Cloud Comput.*, vol. 9, no. 4, pp. 1376–1390, Oct. 2021.

[3] A. Ali, S. A. A. Shah, T. Al Shloul, M. Assam, Y. Yasin, S. Lim, and A. Zia, "Multiobjective Harris hawks optimization-based task scheduling in cloud-fog computing," *IEEE Internet Things J.*, vol. 11, no. 13, pp. 24334–24352, Jul. 2024.

[4] H. Zhang and R. Jia, "Application of chaotic cat swarm optimization in cloud computing multi objective task scheduling," *IEEE Access*, vol. 11, pp. 95443–95454, 2023.

[5] S. V. A. Kumer, N. Prabakaran, E. Mohan, B. Natarajan, G. Sambasivam, and V. B. Tyagi, "Enhancing cloud task scheduling with a robust security approach and optimized hybrid POA," *IEEE Access*, vol. 11, pp. 122426–122445, 2023.

[6] X. Tang, W. Cao, H. Tang, T. Deng, J. Mei, Y. Liu, C. Shi, M. Xia, and Z. Zeng, "Cost-efficient workflow scheduling algorithm for applications with deadline constraint on heterogeneous clouds," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 9, pp. 2079–2092, Sep. 2022.

[7] J. Liu, J. Ren, W. Dai, D. Zhang, P. Zhou, Y. Zhang, G. Min, and N. Najjari, "Online multi-workflow scheduling under uncertain task execution time in IaaS clouds," *IEEE Trans. Cloud Comput.*, vol. 9, no. 3, pp. 1180–1194, Jul. 2021.

[8] J. Lou, Z. Tang, S. Zhang, W. Jia, W. Zhao, and J. Li, "Cost-effective scheduling for dependent tasks with tight deadline constraints in mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 10, pp. 5829–5845, Aug. 2022.

[9] J. Tang, M. M. Jalalzai, C. Feng, Z. Xiong, and Y. Zhang, "Latency-aware task scheduling in software-defined edge and cloud computing with erasure-coded storage systems," *IEEE Trans. Cloud Comput.*, vol. 11, no. 2, pp. 1575–1590, Sep. 2022.

[10] K. Sharma, M. M. Hassan, S. K. Pandey, M. Saini, R. Doriya, M. K. Ojha, A. Sinha, C. Kaushal, A. K. Bairagi, and N. F. Soliman, "Cloud based multi-robot task scheduling using PMW algorithm," *IEEE Access*, vol. 11, pp. 146003–146013, 2023.

[11] D. Alsadie, "A Metaheuristic framework for dynamic virtual machine allocation with optimized task scheduling in cloud data centers," *IEEE Access*, vol. 9, pp. 74218–74233, 2021.

[12] L. Ye, Y. Xia, L. Yang, and Y. Zhan, "Dynamic scheduling stochastic multiworkflows with deadline constraints in clouds," *IEEE Trans. Autom. Sci. Eng.*, vol. 20, no. 4, pp. 2594–2606, Jul. 2022.
[13] I. Attiya, M. A. Elaziz, L. Abualigah, T. N. Nguyen, and A. A. A. El-Latif, "An improved hybrid swarm intelligence for scheduling IoT application tasks in the cloud," *IEEE Trans. Ind. Informat.*, vol. 18, no. 9, pp. 6264–6272, Sep. 2022.
[14] I. M. Ali, K. M. Sallam, N. Moustafa, R. Chakraborty, M. Ryan, and K. R. Choo, "An automated task scheduling model using non-dominated sorting genetic algorithm II for fog-cloud systems," *IEEE Trans. Cloud Comput.*, vol. 10, no. 4, pp. 2294–2308, Oct. 2022.
[15] L. Zhu, K. Huang, Y. Hu, and X. Tai, "A self-adapting task scheduling algorithm for container cloud using learning automata," *IEEE Access*, vol. 9, pp. 81236–81252, 2021.
[16] S. K. Paul, S. K. Dhal, S. K. Majhi, A. Mahapatra, P. K. Gantayat, and S. Panda, "Optimizing task scheduling and resource utilization in cloud environment: A novel approach combining pattern search with artificial rabbit optimization," *IEEE Access*, vol. 12, pp. 67130–67148, 2024.
[17] H. Mahmoud, M. Thabet, M. H. Khafagy, and F. A. Omara, "Multiobjective task scheduling in cloud environment using decision tree algorithm," *IEEE Access*, vol. 10, pp. 36140–36151, 2022.
[18] A. Ali and M. M. Iqbal, "A cost and energy efficient task scheduling technique to offload microservices based applications in mobile cloud computing," *IEEE Access*, vol. 10, pp. 46633–46651, 2022.
[19] Y. Wang and X. Zuo, "An effective cloud workflow scheduling approach combining PSO and idle time slot-aware rules," *IEEE/CAA J. Autom. Sinica*, vol. 8, no. 5, pp. 1079–1094, May 2021.
[20] X. Tang, "Reliability-aware cost-efficient scientific workflows scheduling strategy on multi-cloud systems," *IEEE Trans. Cloud Comput.*, vol. 10, no. 4, pp. 2909–2919, Oct. 2022.
[21] N. K. Walia, N. Kaur, M. Alowaidi, K. S. Bhatia, S. Mishra, N. K. Sharma, S. K. Sharma, and H. Kaur, "An energy-efficient hybrid scheduling algorithm for task scheduling in the cloud computing environments," *IEEE Access*, vol. 9, pp. 117325–117337, 2021.
[22] K. Li, "Design and analysis of heuristic algorithms for energy-constrained task scheduling with device-edge-cloud fusion," *IEEE Trans. Sustain. Comput.*, vol. 8, no. 2, pp. 208–221, Jul. 2022.
[23] S. Mangalampalli, S. S. Hashmi, A. Gupta, G. R. Karri, K. V. Rajkumar, T. Chakrabarti, P. Chakrabarti, and M. Margala, "Multi objective prioritized workflow scheduling using deep reinforcement based learning in cloud computing," *IEEE Access*, vol. 12, pp. 5373–5392, 2024.
[24] B. Saemi, A. A. R. Hosseinabadi, A. Khodadadi, S. Mirkamali, and A. Abraham, "Solving task scheduling problem in mobile cloud computing using the hybrid multi-objective Harris hawks optimization algorithm," *IEEE Access*, vol. 11, pp. 125033–125054, 2023.
[25] L. Ye, L. Yang, Y. Xia, and X. Zhao, "A cost-driven intelligence scheduling approach for deadline-constrained IoT workflow applications in cloud computing," *IEEE Internet Things J.*, vol. 11, no. 9, pp. 16033–16047, May 2024.
[26] W. Fan, L. Zhao, X. Liu, Y. Su, S. Li, F. Wu, and Y. Liu, "Collaborative service placement, task scheduling, and resource allocation for task offloading with edge-cloud cooperation," *IEEE Trans. Mobile Comput.*, vol. 23, no. 1, pp. 238–256, Oct. 2022.
[27] C.-C. Hu, "Profit-based algorithm of joint real-time task scheduling and resource allocation in C-RANs," *IEEE Internet Things J.*, vol. 8, no. 2, pp. 941–950, Jan. 2021.
[28] X. Ma, H. Xu, H. Gao, and M. Bian, "Real-time multiple-workflow scheduling in cloud environments," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 4, pp. 4002–4018, Dec. 2021.
[29] S. Sudheer Mangalampalli, G. Reddy Karri, S. Nandan Mohanty, S. Ali, A. M. Alamri, and S. A. Alqahtani, "Efficient hybrid DDPG task scheduler for HPC and HTC in cloud environment," *IEEE Access*, vol. 12, pp. 108897–108920, 2024.
[30] L. Ye, Y. Xia, L. Yang, and C. Yan, "SHWS: Stochastic hybrid workflows dynamic scheduling in cloud container services," *IEEE Trans. Autom. Sci. Eng.*, vol. 19, no. 3, pp. 2620–2636, Jul. 2022.
[31] S. Sahoo, B. Sahoo, and A. K. Turuk, "A learning automata-based scheduling for deadline sensitive task in the cloud," *IEEE Trans. Services Comput.*, vol. 14, no. 6, pp. 1662–1674, Nov. 2021.
[32] H. Yuan, G. Tang, X. Li, D. Guo, L. Luo, and X. Luo, "Online dispatching and fair scheduling of edge computing tasks: A learning-based approach," *IEEE Internet Things J.*, vol. 8, no. 19, pp. 14985–14998, Oct. 2021.
[33] A. Liu, M. Gao, and J. Tang, "Multi-mode instance-intensive workflow task batch scheduling in containerized hybrid cloud," *IEEE Trans. Cloud Comput.*, vol. 12, no. 1, pp. 159–173, Jan. 2024.
[34] S. Lipsa, R. K. Dash, N. Ivkovic, and K. Cengiz, "Task scheduling in cloud computing: A priority-based heuristic approach," *IEEE Access*, vol. 11, pp. 27111–27126, 2023.
[35] K. Li, L. Jia, and X. Shi, "Research on cloud computing task scheduling based on PSOMC," *J. Web Eng.*, vol. 21, no. 6, pp. 1749–1766, Nov. 2022.
[36] F. Yao, C. Pu, and Z. Zhang, "Task duplication-based scheduling algorithm for budget-constrained workflows in cloud computing," *IEEE Access*, vol. 9, pp. 37262–37272, 2021.
[37] Z. Cui, T. Zhao, L. Wu, A. K. Qin, and J. Li, "Multi-objective cloud task scheduling optimization based on evolutionary multi-factor algorithm," *IEEE Trans. Cloud Comput.*, vol. 11, no. 4, pp. 3685–3699, Oct. 2023.
[38] B. Kruekaew and W. Kimpan, "Multi-objective task scheduling optimization for load balancing in cloud computing environment using hybrid artificial bee colony algorithm with reinforcement learning," *IEEE Access*, vol. 10, pp. 17803–17818, 2022.
[39] Y. Laili, F. Guo, L. Ren, X. Li, Y. Li, and L. Zhang, "Parallel scheduling of large-scale tasks for industrial cloud–edge collaboration," *IEEE Internet Things J.*, vol. 10, no. 4, pp. 3231–3242, Feb. 2023.
[40] P. Banerjee, S. Roy, A. Sinha, M. M. Hassan, S. Burje, A. Agrawal, A. K. Bairagi, S. Alshathri, and W. El-Shafai, "MTD-DHJS: Makespan-optimized task scheduling algorithm for cloud computing with dynamic computational time prediction," *IEEE Access*, vol. 11, pp. 105578–105618, 2023.
[41] N. Balani, P. Chavan, and M. Ghonghe, "Design of high-speed blockchain-based sidechaining peer to peer communication protocol over 5G networks," *Multimedia Tools Appl.*, vol. 81, no. 25, pp. 36699–36713, Oct. 2022.
[42] C.-Q. Dai, Y. Zhang, F. R. Yu, and Q. Chen, "Dynamic scheduling scheme with task laxity for data relay satellite networks," *IEEE Trans. Veh. Technol.*, vol. 73, no. 2, pp. 2605–2620, Feb. 2024.
[43] N. Balani and P. Chavan, "Design of heuristic model to improve blockchain-based sidechain configuration," *Int. J. Comput. Sci. Eng.*, vol. 26, no. 4, pp. 372–384, 2023.
[44] Y. Yang, H. Shen, and H. Tian, "Scheduling workflow tasks with unknown task execution time by combining machine-learning and greedy-optimization," *IEEE Trans. Services Comput.*, vol. 17, no. 3, pp. 1181–1195, May 2024.
[45] N. Balani and P. Chavan, "CSIMH: Design of an efficient security-aware customized sidechaining model via iterative meta-heuristics," *J. Appl. Secur. Res.*, vol. 19, no. 3, pp. 453–493, Jul. 2024.



NEMILIDINEA ANANTHARAMI REDDY received the B.Tech. degree from JNTUK, in 2013, and the M.Tech. degree from the K. L. Deemed to be University, in 2021. He is currently pursuing the Ph.D. degree with the School of Computer Science and Engineering (SCOPE), VIT-AP University, Andhra Pradesh, India. His research interests include cloud computing, fog computing, machine learning, deep learning, and reinforcement learning.



B.V. GOKULNATH is currently an Assistant Professor Senior Grade 1 with the School of Computer Science Engineering, VIT-AP University, Andhra Pradesh, Amaravati. His current research interests include machine learning, deep learning, image processing, data science, artificial intelligence, computer vision, predictive analytics, and telemedicine.